

8. Data Abstraction and Encapsulation

★ Encapsulation -

Encapsulation in Java is a process of wrapping code and data together into a single unit, for example, a capsule which is mixed of several medicines.

We can create a fully encapsulated class in Java by making all the data members of the class private. Now we can use setter and getter methods to set and get the data in it.

The Java Bean class is the example of a fully encapsulated class.

Data Encapsulation is a technique, feature of object oriented programming that suggests to encapsulate (bind) the data (variables) and data operations (methods) together in a class framework so that the operations on data are performed quickly/speedily, the logic will be hidden from the end users and the data becomes inaccessible directly.

Data Encapsulation basically works for the binding of data and the methods.

The concepts of data encapsulation are implemented in every java program by constructing the class, defining the variables in a class, constructing the methods in a class to perform operations on to the variables making the variables or methods as private/protected for data hiding.

Classes are generally build and implemented for the implementation of data abstraction and data encapsulation.

★ Data Abstraction-

Data abstraction is a technique of oop's that allows the user to define the data members in more secured, safely format so that other users cannot access the data.

Data abstraction means abstract, hide the data, provide protection, safety to the data.

Data abstraction recommends that -

Do not allow the users to interact directly with the data, data members. Allow the users to interact with the data through the method (services).

Concept of data abstraction are implemented in java through following ways -

- I) Access Specifiers / modifiers.
- II) Abstract classes and methods.
- III) Interfaces.

I) Access Specifiers / Modifiers -

The access modifiers in Java specifies the accessibility or scope of a field, method, constructor or class.

There are four types of Access modifiers -

1) Private -

The access level of a private modifier is only within the class.

2) Default -

The access level of a default modifier is only within the package. If you do not specify any access level, it will be the default.

3) Protected -

The access level of a protected modifier is within the package and outside the package through child class. If you do not make child class, it cannot be accessed from outside the package.

4) Public -

The access level of a public modifier is everywhere.

There are many non-access modifiers, such as static, abstract, synchronized, native, volatile, transient, etc.

Local Variables

Private variables / methods

Security Default Protected Variables / methods / classes

Protected Default variables / methods / classes

Public Variables / methods / classes

Local variables are the most secured variables, Java gives first priority to local then private then protected then default and at last public.

Example -

```
class Student
{
```

```
    private int rollno;
```

```
    private String name, course;
```

```
    double permarks;
```

```
void readdata(C)
```

```
{
```

```
}
```

```
void display(C)
```

```
{
```

```
}
```

```
}
```

In above example - variables are defined as private, so that such a variables (data members) and their values (data) cannot be accessed outside of the class. Such a variables can be accessed only through the methods `readdata()` and `display()`. So that variables and their data becomes safe, secured. The concepts of data abstraction are implemented through access modifier "Private".

Data abstraction can also be implemented using protected in inheritance so that data and data members can be accessible only in super class or sub class, not from rest of part.

Data abstraction also recommends to define the variables (data members) inside the methods so that the variables are created as local variables which can not be accessible from other methods, other classes.

★ Data Hiding -

Data hiding means to define the data members of class using private or protected access modifiers so that the values (data) of such variables cannot be accessible for the users so that the data stored in variables will be hidden.

Data hiding is basically used to abstract the data members and the methods that processes the data. Methods are hidden by making them as abstract. Methods can be made private or protected.

Example -

```
class Student
{
    private int rollno;
    private String name;
    private int m1, m2, m3, m4, m5, tot-marks;
    private double per-marks;
    void readData()
    {
        // code to read Student's rollno, name & marks.
        result();
    }
    private void result()
    {
        // code to calculate total marks, percentage.
    }
    void display()
    {
        // code to display resultant data
    }
}
```

★ Coupling -

Coupling means bind the data and methods together, Bind more than one methods together, Bind more than one classes together.

When class will be constructed and variables, methods are defined inside the class, automatically the variables and methods are binded, coupled together.

In case of method overriding, the method of super class and the method of sub class are binded, coupled together.

In case of inheritance, base class and derived class are binded, coupled together.

In case of interface and class, when class implements to interface, interface and class are coupled together.

There are two types of Coupling -

- 1) loose coupling or loosely coupled
- 2) tightly coupled or strongly coupling.

When data and methods or multiple methods or multiple class or interface - classes are binded, coupled together in such a way that - changes made in one side will not reflect in other side such a kind of coupling is known as loosely coupled.

If changes made in one side will reflect in other side then such a kind of coupling is referred as strongly or tightly coupled.

II) Abstract classes and Methods -

Abstract class -

A class which is defined using a abstract keyword and that class cannot be instantiated (cannot create an object of that class). That should be extended (inherited) by other sub class. Such a class is known as Abstract class.

Example -

```
abstract class Student  
{
```

```
}
```

Remember that -

- 1) We cannot create an object of abstract class.
- 2) Abstract class always acts as base class (super class) for its implementation.
- 3) We have to use abstract class in inheritance as a super class.
- 4) If there is an abstract method in a class, class must be abstract.
- 5) Inside the abstract class, we can define abstract methods as well as regular methods (non abstract methods).

Abstract Method-

The method which is created using abstract keyword, which is just defined, which doesn't have a body, which should be overridden from the sub class, such a method is known as abstract method.

Example-

```
abstract void result();
```

```
abstract void display (int x, int y);
```

Remember that-

- 1) Abstract method always begins with abstract keyword.
- 2) Abstract method is just defined, it doesnot have body.
- 3) Abstract methods are Compulsory overided from sub class. (or you should make this class as abstract).
- 4) Abstract method may be parameterized or non-parameterized.
- 5) Abstract methods are used to inform the program to include such a kind of method in sub class with specified prototype.

Example -

```
abstract class Product
```

```
{  
    abstract void show();
```

```
    void display();  
}
```

```
    System.out.println("Display method of Product class");  
}
```

```
class ProductData extends Product
```

```
{
```

```
    void show();
```

```
{
```

```
    System.out.println("Show method of abstract class  
Product");
```

```
}
```

```
    void myResult();
```

```
{
```

```
        display();
```

```
        System.out.println("My Result method of  
ProductData class");
```

```
}
```

```
}
```

```
public class AbstractDemo
```

```
{
```

```
    public static void main(String args[])
```

```
{
```

```
        ProductData pd = new ProductData();
```

```
        pd.show();
```

```
        pd.myResult();
```

```
}
```

```
}
```

III) Interfaces-

An interface in Java is a blueprint of a class. It has static constants and abstract methods.

Syntax-

```
interface <interface name>
{
    declaration of variables;
    declaration of methods;
}
```

Points to Remember-

- 1) The interface in Java is a mechanism to achieve abstraction. There can be only abstract methods in the Java interface, not method body. It is used to achieve abstraction and multiple inheritance in Java.
- 2) Interface also represents the IS-A relationship.
- 3) It cannot be instantiated just like the abstract class.
- 4) Since Java 8, we can have default and static methods in an interface.
- 5) Since Java 9, we can have private methods in an interface.

6) It can be used to achieve loose coupling.

7) Declaration-

An interface is declared by using the "interface" keyword.

8) A class that implements an interface must implement all the methods declared in the interface.

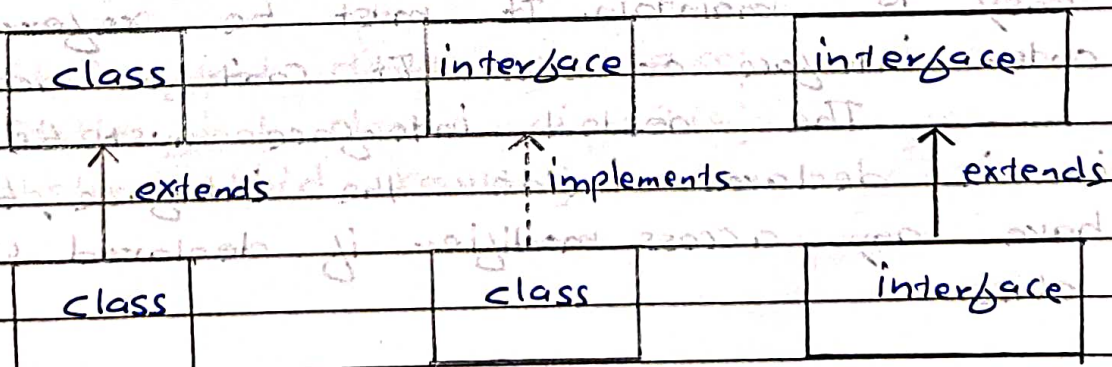
9) Java 8 Interface Improvements-

Since Java-8, interface can have default and static methods which is discussed later.

10) The Java Compiler adds public and abstract keywords before the interface method. Moreover, it adds public, static, and final keywords before data members.

In other words, Interface fields are public, static and final by default and the methods are public and abstract.

11) A class extends another class, an interface extends another interface, but a class implements an interface.



12) Multiple inheritance in Java by interface-

If a class implements multiple interfaces or an interface extends multiple interfaces, it is known as multiple inheritance.

13) Since Java 8, we can have method body in interface. But we need to make it default method. Since Java 8, we can have static and default method in interface.

★ Marker or tagged Interface-

An interface which has no member is known as a marker or tagged interface.

example-

Serializable, Cloneable, etc.

They are used to provide some essential information to JVM so that JVM can perform some useful operation.

★ Nested Interface-

An interface declared within another interface or class, is known as a nested interface. The nested interfaces are used to group related interfaces so that they can be easy to maintain. It must be referred to by the outer interface or class. It can't be accessed directly.

The nested interface must be public if it is declared inside the interface, but it can have any access modifier if declared within the class.

Nested interfaces are declared static. We can also define a class inside interface, the Java Compiler creates a static nested class.

Examples -

```
1) interface Shape
```

```
{
    interface Circle
```

```
{
    }
}
```

Interfaces are divided into 2 Categories -

- 1) Predefined or inbuilt interfaces.
- 2) User defined interfaces.

Examples -

1) Write a java program using interface.

Write a java program to find the area of circle using interface.

⇒

```
interface mydata
```

```
{
    static final double r = 5.65;
```

```
    static final double pi = 3.14;
```

```
    abstract void result();
```

```
}
```

```
class Circle implements mydata
```

```
{
```

```
    double a;
```

```
    public void result()
```

```
{
```

```
        a = pi * r * r;
```

```
    }
```

```
    void display()
```

```
{
```

```
        System.out.println("Radius of circles " + r);
```

```
        System.out.println("Area of circle = " + a);
```

```
    }
```

```
}
```

```
public class InterfaceDemo
```

```
{
```

```
    public static void main (String args[])
```

```
{
```

```
        Circle c = new Circle();
```

```
        c.result();
```

```
        c.display();
```

```
    }
```

```
}
```

- 2) Write a java program for multiple inheritance.
Write a java program using multiple interfaces that are implemented through the single/same class.
(multiple interfaces and only one class)



```
interface Odd-Even
{
```

```
    static final int x = 45;
```

```
    abstract void check-oe();
}
```

```
interface Pos-neg-zero
{
```

```
    static final double y = 25.65;
```

```
    abstract void check-phz();
}
```

```
class checking implements Odd-Even, pos-neg-zero
{
```

```
    public void check-oe()
    {
```

```
        System.out.println("Number x = " + x);
```

```
        if (x % 2 == 0)
```

```
            System.out.println("Number " + x + " is even");
```

```
        else
```

```
            System.out.println("Number " + x + " is odd");
```

```
    }
```

```
    public void check-phz()
    {
```

```
        System.out.println("In Number y = " + y);
```

```
        if (y > 0)
```

```
            System.out.println("y + " is positive");
```

```
        else if (y < 0)
```

```
            System.out.println("y + " is negative");
```

```
        else
```

```
            System.out.println("y + " is zero");
```

```

    }
}

public class InterfaceDemo
{
    public static void main (String args[])
    {
        Checking obj = new Checking();
        obj.check-ae();
        obj.check-prime();
    }
}

```

★ Defining the reference of Interface-

We cannot create an instance (Object) of interface but we can create the reference (reference variable) of interface.

example -

```
mydata ref;
```

Where mydata is interface and ref is its reference (reference variable).

Note -

In reference of interface, we can assign the reference of class.

example -

```
mydata ref = new Circle();
```

Where mydata is interface and Circle is class which implements mydata interface.

★ this keyword in Java -

this keyword is used as a reference of current object.

Usage of this keyword -

- 1) this can be used to refer current class instance variable. -

If there is ambiguity between the instance variables and parameters, this keyword resolves the problem of ambiguity.

- 2) this can be used to invoke current class method (implicitly) -

If you don't use the this keyword, compiler automatically adds this keyword while invoking the method present in same class.

- 3) this() can be used to invoke current class constructor.

The this() constructor call can be used to invoke the current class constructor. It is used to reuse the constructor. In other words, it is used for constructor chaining.

Rule - Call to this() must be the first statement in constructor.

- 4) this can be passed as an argument in the method call -

It is mainly used in the event handling. In event handling (or) in a situation where we have

to provide reference of a class to another one. It is used to reuse one object in many methods.

5) `this` can be passed as argument in the Constructor call—

It is useful if we have to use one object in multiple classes.

6) `this` can be used to return the current class instance from the method—

In such case, return type of the method must be the class type (non-primitive).

Example—

```
class MyClass
```

```
{
    int a, b;
```

```
    MyClass ()
```

```
{
    System.out.println("I am default Constructor");
```

```
    a=100;
```

```
    b=200;
```

```
    System.out.println("Initial values are a=" + a + " + " + b + " + " + b);
```

```
}
```

```
    MyClass (int x, int y)
```

```
{
```

```
    this ();
```

```
    System.out.println("I am parameterized Constructor");
```

```
    a=x;
```

```

b=y;
System.out.println("Initial Values : a=" + a + " " +
                    "b=" + b);

```

```

}
void result()
{

```

```

    int add = a+b;
    System.out.println("Addition = " + add);
}
}

```

```

class ThisDemo
{

```

```

    public static void main(String args[])
    {

```

```

        MyClass obj = new MyClass();

```

```

        obj.result();

```

```

        System.out.println();

```

```

        MyClass obj1 = new MyClass(40, 50);

```

```

        obj1.result();
    }
}

```

output-

I am default Constructor

Initial Values : a = 100 b = 200

Addition = 300

I am default Constructor

Initial Values : a = 100 b = 200

I am parameterized Constructor

Initial Values : a = 40 b = 50

Addition = 90

★ Instance initializer block -

Instance initializer block is used to initialize the instance data members. It runs each time when object of class is created.

Example -

```
class Bike
{
    int speed;
    Bike()
    {
        System.out.println("Constructor is invoked");
    }
    {
        System.out.println("Instance initializer block invoked");
    }
    public static void main (String args[])
    {
        Bike b1 = new Bike();
        Bike b2 = new Bike();
    }
}
```

Output -

Instance initializer block invoked.

Constructor is invoked.

Instance initializer block invoked.

Constructor is invoked.

In above example, it seems that instance initializer block is firstly invoked, but No. Instance initializer block is invoked at the time

of object creation. The java Compiler copies the instance initializer block in the constructor after the first statement `Super()`. So firstly constructor is invoked.

Rules for Instance Initializer block -

- 1) The instance initializer block is created when instance of the class is created.
- 2) The instance initializer block is invoked after the parent class constructor is invoked (i.e. after `Super()` constructor call).
- 3) The instance initializer block comes in the order in which they appear.